

m/ code you can set up the basic Python code to send the Slack notification. Here is an example from our gitlabsnowflakenotification lambda code:

```
python
import urllib3
import json
import os

http = urllib3.PoolManager()

def lambdahandler(event, context):
    url = os.environ["slackwebhookurl"]
    data = event["Records"][0]
    timestamp = data["Sns"]["Timestamp"]
    timestamp = timestamp.replace("T", " ").replace("Z", "")
    taskdetails= json.loads(data["Sns"]["Message"])
    if 'taskName' in taskdetails:
        failuretypename= taskdetails['taskName']
        errormeessage=taskdetails['messages'][0]['errorMessage']
        title="Snowflake Task Failure Alert"
    if 'pipeName' in taskdetails:
        failuretypename= taskdetails['pipeName']
        errormeessage=taskdetails['messages'][0]['firstError']
        title="Snowpipe Failure Alert"

    errormessagemarkdown=f"{errormeessage}"

    loglink='https://gitlab.com/gitlab-
data/runbooks/-/blob/main/triageissues/snowflakepipelinefailuretriage.md'
    loglinkmarkdown = f"<{loglink}|Runbook>"

    message = {
        "channel": "data-pipelines",
        "username": "SNOWFLAKETASKPIPE",
        "iconemoji": "snowflake",
        "attachments": [{
            "fallback": "Snowflake Task Failure Alert",
            "color": "FF0000",
            "fields": [
                {
                    "title": title,
                    "value": failuretypename
                },
                {
                    "title": "Timestamp",
                    "value": timestamp
                },
                {
                    "title": "Error Message",
                    "value": errormessagemarkdown
                }
            ]
        }
    ]
}
```

```

        },
        {
            "title": "Steps to view error message in snowflake",
            "value": loglinkmarkdown
        }
    ]
}

encodedmsg = json.dumps(message).encode("utf-8")
resp = http.request("POST", url, body=encodedmsg)

print(
    {
        "message": event['Records'],
        "statuscode": resp.status,
        "response": resp.data
    }
)

```

Things to consider during the implementation:

- Good practice is to encrypt environment variables you plan to use in AWS Lambda function, as a part of good practices. Here is the comprehensive guideline on how to secure environment variables.

Above function will send all the event notification information into Slack for snowflake tasks and snowpipe failures and it will look like:

Step 7: Add trigger to the Lambda function as the SNS topic which is created in step 1

This will trigger the lambda function which will send the Slack notification

Step 8: Alter snowflake Snowpipe and Snowflake task to send notification on failure

In this example I have done it for one of snowflake task

```

sql
ALTER TASK RAW.PROMETHEUS.PROMETHEUSLOADTASK suspend;
ALTER TASK RAW.PROMETHEUS.PROMETHEUSLOADTASK SET ERRORINTEGRATION = gitlabdataan
ALTER TASK RAW.PROMETHEUS.PROMETHEUSLOADTASK resume;

```

The task should be suspended in order to apply the integration.

Once above is defined on any failure the notification will be sent to Slack channel.

All the required permission in AWS account has been provided to all data platform team member. Also the Permission has been granted to loader role to use snowflake integration.

Note: Snowpipe error notifications only work when the ONERROR copy option is set to SKIPFILE (the default value). Snowpipe will not send any error notifications if the ONERROR copy option is set to CONTINUE.

You can use the NOTIFICATIONHISTORY table function to query the history of notifications sent through Snowpipe. For more information, refer to NOTIFICATIONHISTORY documentation.

Triage errors

When we got an alert in Slack, should go to a runbooks page and analyze the issue.

SECTION: enterprise-data/platform/snowplow/index.md

Snowplow Overview

Snowplow is an open source, event analytics platform. There is a business entity that runs SaaS for Snowplow and they also maintain the code for the open source product. The general architecture overview of snowplow is on GitHub and has more detail on the basics of how it works and how it is set up.

In June of 2019, we switched sending Snowplow events from a third party to sending them to infrastructure managed by GitLab, documented on this page. From the perspective of the data team, not much changed from the third party implementation. Events are sent through the collector and enricher and dumped to S3.

As of December 2024, we have switched over to a new Snowplow env called aws-snowplow-prd, more information is detailed in the Snowplow internal handbook.

Snowplow - adding new appid

When new application should be tracked by Snowplow here is the few things should be considered.

The right appid, and collector URL should be done in coordination with the data team. URL will stay the same snowplowprd.trx.gitlab.net. Any appid is fine if there are no other concerns around enabling tracking on CustomersPortal staging as well.

> Note: Any un-expected events (with wrong appid) are normally dropped.

The only model with appid filtering is snowplowbaseevents. That model flows downstream to the page view models:

mermaid
flowchart LR

SPBE[snowplowbaseevents] --> SPWE[snowplowwebevents]
SPWE --> snowplowpageviews
snowplowpageviews --> commonprep.prepsnowplowpageviewsall
commonprep.prepsnowplowpageviewsall --> common.fctbehaviourwebsitepageviews
snowplowpageviews --> legacy.snowplowpageviews

In order to add a new appid to snowplowbaseevents (and downstream page view models), the snowplow:appids variable in the dbt package must be updated. Those values are set in the dbtproject.yml file. As an example, here is an issue to update the variable.

GitLab Implementation

The original design document to move our Snowplow infrastructure from a 3rd-party hosting service to 1st-part is documented in the Infrastructure design library. This was written before the build was started and contains many of the assumptions and design decisions.

Snowplow is built with Terraform on AWS documented in the config-mgmt project.

For a detailed walk-through of our setup, watch this GitLab Unfiltered internal video.

S3

Enriched events are stored in TSV format in the bucket s3://gitlab-com-snowplow-events/output/. Bad events are stored as JSON in s3://gitlab-com-snowplow-events/enriched-bad/. For both buckets, there are paths that follow a date format of /YYYY/MM/DD/HH/<data>.

Data Warehousing

<details><summary>Click to expand</summary>

Snowpipe

Once events are available in S3, we ingest them into the data warehouse using Snowpipe. This is a feature of our Snowflake Data Warehouse.

An Amazon SQS event queue was set up for the good and bad event paths.

To run properly, Snowpipe needs a "stage" in Snowflake and a table to write to.

The good and bad S3 paths each have their own Stage within Snowflake.

These are named gitlabevents and gitlabbadevents, respectively. They are owned by the LOADER role.

The create table statements for the good and bad events are as follows:

```
sql
-- Good Events
CREATE OR REPLACE TABLE snowplow.gitlabevents
```

(
appid VARCHAR,
platform VARCHAR,
etltstamp VARCHAR,
collectortstamp VARCHAR,
dvcecreatedtstamp VARCHAR,
event VARCHAR,
eventid VARCHAR,
txnid VARCHAR,
nametracker VARCHAR,
vtracker VARCHAR,
vcollector VARCHAR,
vetl VARCHAR,
userid VARCHAR,
useripaddress VARCHAR,
userfingerprint VARCHAR,
domainuserid VARCHAR,
domainsessionidx VARCHAR,
networkuserid VARCHAR,
geocountry VARCHAR,
georegion VARCHAR,
geocity VARCHAR,
geozipcode VARCHAR,
geolatitude VARCHAR,
geolongitude VARCHAR,
georegionname VARCHAR,
ipisp VARCHAR,
iporganization VARCHAR,
ipdomain VARCHAR,
ipnetspeed VARCHAR,
pageurl VARCHAR,
pagetitle VARCHAR,
pagerereferrer VARCHAR,
pageurlscheme VARCHAR,
pageurlhost VARCHAR,
pageurlport VARCHAR,
pageurlpath VARCHAR,
pageurlquery VARCHAR,
pageurlfragment VARCHAR,
refrurlscheme VARCHAR,
refrurlhost VARCHAR,
refrurlport VARCHAR,
refrurlpath VARCHAR,
refrurlquery VARCHAR,
refrurlfragment VARCHAR,
refrmedium VARCHAR,
refrsource VARCHAR,
refrterm VARCHAR,
mktmedium VARCHAR,
mktsource VARCHAR,

mktterm	VARCHAR,
mktcontent	VARCHAR,
mktcampaign	VARCHAR,
contexts	VARCHAR,
secategory	VARCHAR,
seaction	VARCHAR,
selabel	VARCHAR,
seproperty	VARCHAR,
sevalue	VARCHAR,
unstruetevent	VARCHAR,
trorderid	VARCHAR,
traffiliation	VARCHAR,
trtotal	VARCHAR,
trtax	VARCHAR,
trshipping	VARCHAR,
trcity	VARCHAR,
trstate	VARCHAR,
trcountry	VARCHAR,
tiorderid	VARCHAR,
tisku	VARCHAR,
tiname	VARCHAR,
ticategory	VARCHAR,
tiprice	VARCHAR,
tiquantity	VARCHAR,
ppxoffsetmin	VARCHAR,
ppxoffsetmax	VARCHAR,
ppyoffsetmin	VARCHAR,
ppyoffsetmax	VARCHAR,
useragent	VARCHAR,
brname	VARCHAR,
brfamily	VARCHAR,
brversion	VARCHAR,
brtype	VARCHAR,
brrenderengine	VARCHAR,
brlang	VARCHAR,
brfeaturespdf	VARCHAR,
brfeaturesflash	VARCHAR,
brfeaturesjava	VARCHAR,
brfeaturesdirector	VARCHAR,
brfeaturesquicktime	VARCHAR,
brfeaturesrealplayer	VARCHAR,
brfeatureswindowsmedia	VARCHAR,
brfeaturesgears	VARCHAR,
brfeaturesilverlight	VARCHAR,
brcookies	VARCHAR,
brcolordepth	VARCHAR,
brviewwidth	VARCHAR,
brviewheight	VARCHAR,
osname	VARCHAR,
osfamily	VARCHAR,

```

osmanufacturer    VARCHAR,
ostimezone        VARCHAR,
dvcetype          VARCHAR,
dvceismobile      VARCHAR,
dvcescreenwidth   VARCHAR,
dvcescreenheight  VARCHAR,
doccharset        VARCHAR,
docwidth          VARCHAR,
docheight         VARCHAR,
trcurrency        VARCHAR,
trtotalbase       VARCHAR,
trtaxbase         VARCHAR,
trshippingbase    VARCHAR,
ticurrency        VARCHAR,
tipricebase       VARCHAR,
basecurrency      VARCHAR,
geotimezone       VARCHAR,
mktclickid        VARCHAR,
mktnetwork        VARCHAR,
etltags           VARCHAR,
dvcesenttstamp    VARCHAR,
refrdomainuserid  VARCHAR,
refrdvcetstamp    VARCHAR,
derivedcontexts   VARCHAR,
domainsessionid   VARCHAR,
derivedtstamp     VARCHAR,
eventvendor       VARCHAR,
eventname         VARCHAR,
eventformat       VARCHAR,
eventversion      VARCHAR,
eventfingerprint  VARCHAR,
truetstamp        VARCHAR,
uploadedat        TIMESTAMPNTZ(9) DEFAULT CAST(CURRENTTIMESTAMP() AS TIMESTAMPNTZ(9))
)

```

-- Bad Events Table

```

CREATE OR REPLACE TABLE snowplow.gitlabbadevents
(
  jsoncontext  VARIANT,
  uploadedat   TIMESTAMPNTZ(9) DEFAULT CAST(CURRENTTIMESTAMP() AS TIMESTAMPNTZ(9))
);

```

Since TSV is not as straight-forward as CSV, a custom file format was created with the following statement:

```

sql
CREATE OR REPLACE FILE FORMAT snowplowtsv TYPE = CSV FIELDDELIMITER = '\t'

```

The actual pipe for good events was created using:

```
sql
CREATE OR REPLACE PIPE raw.snowplow.gitlabgoodeventpipe autoingest= TRUE AS
COPY INTO raw.snowplow.gitlabevents
  FROM (SELECT $1, $2, $3, $4, $5, $6, $7, $8, $9,$10,$11,$12,$13,$14,$15,$16,$17,$18,$19,$20
,$21,$22,$23,$24,$25,$26,$27,$28,$29,$30,$31,$32,$33,$34,$35,$36,$37,$38,$39,$40,$41,$42,$43,$4
4,$45,$46,$47,$48,$49,$50,$51,$52,$53,$54,$55,$56,$57,$58,$59,$60,$61,$62,$63,$64,$65,$66,$67,$
68,$69,$70,$71,$72,$73,$74,$75,$76,$77,$78,$79,$80,$81,$82,$83,$84,$85,$86,$87,$88,$89,$90,$91,
$92,$93,$94,$95,$96,$97,$98,$99,100,101,102,103,104,105,106,107,108,109,110,111,112,113,114,115
,116,117,118,119,120,121,122,123,124,125,126,127,128,129,130,131,CAST(CURRENTTIMESTAMP() AS
TIMESTAMPNTZ(9)) AS uploadedat
  FROM @raw.snowplow.gitlabevents)
FILEFORMAT = (FORMATNAME = 'snowplowtsv')
ONERROR = 'skipfile';
```

This highlights the fact that JSON would be a better format. This could be a future iteration of the infrastructure.

The bad event pipe is created as follows:

```
sql
CREATE OR REPLACE PIPE raw.snowplow.gitlabbadeventpipe autoingest= TRUE AS
COPY INTO raw.snowplow.gitlabbadevents (jsontext)
  FROM @raw.snowplow.gitlabbadevents
  FILEFORMAT = (TYPE = 'JSON');
```

To view pipes:

```
sql
SHOW PIPES IN raw.snowplow;
```

To describe a pipe:

```
sql
DESCRIBE PIPE raw.snowplow.gitlabgoodeventpipe;
```

To pause a running pipe:

```
sql
ALTER PIPE raw.snowplow.gitlabgoodeventpipe SET PIPEEXECUTIONPAUSED = TRUE;
```

To force resume a pipe:

```
sql
```

```
SELECT system$pipeforceresume('raw.snowplow.gitlabgoodeventpipe');
```

To check the status of a pipe

```
sql
SELECT system$pipestatus('raw.snowplow.gitlabgoodeventpipe');
```

To force a refresh of the stage so that snowpipe picks up older events:

```
sql
ALTER PIPE gitlabgoodeventpipe refresh;
```

</details>

dbt

To materialize data from the RAW database to PROD for querying, we have implemented a partitioning strategy within dbt. By default, the snowplow models and the Fishtown snowplow package will write to a schema scoped to the current month in the PREP database. For July 2019, the schema would be snowplow201907.

Within each monthly partition all of the base models and the models generated by the package are written for all events that have a derived timestamp that matches the partition date. Different monthly partitions can be generated by passing in variables to dbt at run time:

```
--vars '{"year": "2019", "month": "01", "part": "201901"}'
```

Some models downstream of the monthly partitions (ex. fctbehaviorstructuredevent and martbehaviorstructuredevent) use the incrementalbackfilldate variable to set the start and end for a backfill. This can be used locally to test backfilling a month of data by using the following variable at run time:

```
--vars '{"incrementalbackfilldate": "{{dbt.stringliteral(\'YYYY-MM-DD\')}}'
```

Backfills are done via Airflow. The dbtsnowplowbackfill DAG will generate a task for each month from July 2018 to the current month.

Do Not Track

Our snowplow tracking configuration and particular implementations respect the Do Not Track (DNT) headers whenever it's present on a user's browser.

Duo data redaction

We only keep Duo free form feedback for 60 days in snowflake. This is managed by the duodataredaction DAG, which runs daily, removing contents of the extendedFeedback attribute in the contexts column for all feedback response Snowplow events in RAW and PREP. This timeline

allows for our full-refresh process to complete, updating all downstream data, within 90 days for compliance.

Snowplow improvement: SQL scripting for issue fixing

To generate a script the issue, do the following things:

1. Open an issue in the project snowplow-fix-scripting
2. Open an MR in the same project
3. Adjust config.yml file to adjust your logic.
4. Run the pipeline ·scripting -> ··generatesql in MR

Pipeline ··generatesql

The ··generatesql job is a manually triggered job in the GitLab CI/CD pipeline that generates SQL scripts based on provided parameters. It runs in the ·scripting stage of the pipeline.

To run this job successfully, the following environment variables must be set:

Required Environment Variables:

DATEFROM: Start date for the data range to process in the format YYYY-MM-DD

DATETO: End date for the data range to process in the format YYYY-MM-DD

Optional Environment Variables

LOGLEVEL: Sets the logging verbosity (defaults to DEBUG if not provided). Allowed values: [DEBUG|INFO|WARNING|ERROR|CRITICAL]

DATABASEPREFIX: Optional prefix for database objects or connections. If value is not provided, then PROD code is generated (RAW, PREP, PROD). Otherwise, enter prefix for the database name i.e. 22822-SNOWPLOW-IMPROVEMENT-SQL-SCRIPTING-FOR-ISSUE-FIXING.

Output structure

bash

scripts/

···RAW

····1backup.sql create backup

····2update.sql fix script to update

····3check.sql check script

····4drop.sql drop backup script

···PREP

····1backup.sql create backup

····2update.sql fix script to update

····3check.sql check script

····4drop.sql drop backup script

···PROD

····1backup.sql create backup

····2update.sql fix script to update

····3check.sql check script

.....4drop.sql drop backup script

Usually, the flow will require the pipeline to be executed twice (not necessarily), once to generate a testing script to be run on the dev DB's, and once to generate a prod script to be run on the production db's.

For testing databases, the parameter DATABASEPREFIX will have a value as a prefix of development databases (ie. 22822-SNOWFLOW-IMPROVEMENT-SQL-SCRIPTING-FOR-ISSUE-FIXING) code will be generated like:

```
sql
...
UPDATE "22822-SNOWFLOW-IMPROVEMENT-SQL-SCRIPTING-FOR-ISSUE-
FIXINGPREP".SNOWFLOW202501.SNOWFLOWUNNESTEDEVENTS
  SET pageurlpath = updateprocedure(puserid=userid)
...
```

For the production database, the parameter DATABASEPREFIX will be skipped. Code will look like:

```
sql
...
UPDATE PREP.SNOWFLOW202501.SNOWFLOWUNNESTEDEVENTS
  SET pageurlpath = updateprocedure(puserid=userid)
...
```

The scripts are generated as an artifact. Once the pipeline is complete, click the download button to retrieve them.

SECTION: enterprise-data/platform/tableau/_index.md

Quick Links	Developer Resources	Communications
Tableau Cloud - GitLab	GitLab Tableau Developer Guide	Internal Slack channel
Tableau eLearning Portal	Tableau Style Guide	External Slack channel
Tableau Customer Portal	Developer Tips and Tricks	
Tableau Status Page	Tableau Embedding to Handbook	

Tableau

Tableau is our Enterprise Business Intelligence tool. It is a leader in the Business Intelligence space. We are applying the Tableau Blueprint to launch our Tableau production environment. The Tableau Blueprint outlines the processes and best practices from thousands of Tableau customers. We will apply these processes and best practices in accordance with GitLab's TeamOps and culture.

<details markdown=1>

<summary>Tableau Terms and Definitions</summary>

- Connections: The method Tableau used to communicate with a source of data.
 - Direct to Source: When Tableau uses a built in connector to communicate with source of data like Snowflake or Google Drive.
 - Virtual Connection: A curated connection where the communication to the source of data has been set by the admin team. Will generally simulate a Direct to Source connection.
 - Published Data Source: A curated connection to a single tableau that is the result of data modeling. It is stored on Tableau Cloud.
 - Connect to A File: A method of getting data from the files in your system into Tableau. This includes Excel files, CSV, PDF, etc.
 - Connect to A Server (Cloud): A method of connecting to a data source. This includes any connection to any online data sources, including Tableau Cloud and Snowflake
- Connection Credentials: The information used to certify connection to a source of data.
 - Embedded: Credentials are saved as part of the connection.
 - Prompt User: Credentials must be input every time the connection is used.
- Connection Freshness: Determines how often a connection will communicate with the source of data.
 - Live: Will send a query to the source of data each time the data is needed.
 - Extract: Will send a query when prompted to the source of data storing the result in custom micro database.
 - Scheduled Extract: An extract hosted on Tableau Cloud that will, on a fixed frequency, communicate with the source of data and update the stored data.
- Data Modeling: The method of combining multiple tables of data into a single result table.
 - Data Warehouse: The query to combine the table is stored in the data warehouse repository and is materialized in the data warehouse.
 - Tableau Data Modeling: Relationships are defined between tables, in the connection interface, directly in Tableau.
 - Tableau Blending: Relationships between data sources defined in the visualization construction interface.
 - Custom SQL: Using SQL within a connection to define the relationships between tables.
- Tableau Objects: The elements that a developer will work with to create visualizations.
 - Project: The folder structure maintained in Tableau Cloud. This is where content permissions are administered.
 - Data Source: A collection of connections and data modeling that results in a single result table that can be published and a stand alone file.
 - Flow: A collection of connections and transformation steps that will typically result in creating one or more Data Sources. A Flow is created using the Tableau application called Prep which performs data transformations. Some individuals may use the works Prep and Flow interchangeably. At GitLab we prefer all transformations go through DBT at the Data Warehouse Level.
- Workbook: The base file time for creating visitations.
 - View: The element that represent a chart or other visualization.
 - Dashboard: A collection of views and layout elements.
 - Story: A collection of views and layout elements where the views can be set to a specific filter or parameter value and displayed in a sequential structure.

</details>

Governance Model

Governance is the combination of controls, roles, and repeatable processes that creates trust and confidence in data and analytics. Both IT and business stakeholders on the project team are responsible for defining data and content governance together. In a successful self-service environment, the appropriate levels of governance create accountability and enable, rather than restrict, access to trusted content for users at GitLab. Governance is a spectrum, different kinds of data and content require different kinds of governance. It's not a one-time effort because skills and expectations will evolve. Periodically evaluating governance processes will allow us to evolve and delegate more responsibilities as new analytical skills and use cases develop.

We use a self-governing model at GitLab. In a self-governing model, there is strong collaboration between IT and business users. Certified content and data sources are available, and one off content is being created regularly by Creators and Explorers. Viewers understand the delineation between certified, one off, and sandbox content states. The process of validation, promotion, and certification is well-defined and well-understood by users of all skill levels. With increasing analytical skills across the organization, the boundaries between the roles of the Modern Analytics Workflow are fluid as users switch from consuming to creating to promoting content with the appropriate level of permissions.

BIOps

Our Tableau self-governing model is administered and enforced in the GitLab Tableau Project using BIOps. The BIOps approach will leverage GitLab's repository, maintainer and code review functionality to administer the governance model. Tableau does not currently have a Git integration so our BIOps is not fully automated and there are some limitations. The README file is coming soon which will describe the BIOps workflow. We will iterate on this approach with the GTM and Finance teams during Q1 and Q2 and adjust and adapt as needed.

Tableau Project Architecture

The Project Architecture in Tableau Cloud is replicated and governed in the GitLab Tableau Project. Please see the GitLab Tableau Project for more details. Below are descriptions of the project folders and a sample of the project architecture found in Tableau Cloud.

The top-level folders in our Tableau Project, and their corresponding levels of governance, include:

- Development: content in this folder intentionally includes no governance, in order to enable users to quickly prototype. As such it should be considered to be sandbox content.
- Production: The Tableau Production folder environment is a designated area where validated, high-quality content is published and accessible to end users for business-critical reporting. Content in this folder has been reviewed and approved by Project Leaders.

<details markdown=1>

<summary>Project and Sub-Project Folder Descriptions</summary>

- Top Level Project Folders: The top level project provides what the purpose of the workbook

is. There are two top level projects; Production, Development. This is the highest folder level that the Tableau user lands on. These folders guide the user either down a Production path to view certified content or development path to view sandbox content.

- 2nd Level Sub-Project Folders: The 2nd level project provides who the primary owner of the workbook is. This level of the architecture contains sub-projects for each department and cross-functional business motion such as the Go To Market Motion. Each department and cross-functional business motion will have their own sub-project. This gives us flexibility to create different types of security at the sub-project level based on specific departmental and business motion needs.

- 3rd Level Sub-Project Folders: The 3rd level project provides security around who can see the workbook. Our SAFE Data Program is applied on this level of the architecture. This allows us flexibility to apply more security controls, by department, in the future to include enhanced SAFE Data program controls, row and column level security, and security around confidential information. Applying the security at this level will allow for customized, and scalable security programs by department and business motion.

</details>

<details markdown=1>

<summary>Project Architecture</summary>

1. Development (Sandbox Environment)

1. Customer Success

1. General

1. Data Source Name

2. Workbook Name

1. SAFE

1. Data Source Name

2. Workbook Name

1. Workbook Name

2. Data Team

3. Engineering

4. Finance

5. Go To Market

6. Marketing

7. People

8. Product

9. Sales

10. Security

1. Production (Maps to our Trusted Data Development Process)

1. Customer Success

1. General

1. Data Source Name

2. Workbook Name

1. SAFE

1. Data Source Name

2. Workbook Name

2. Data Team

3. Engineering

4. Finance
5. Go To Market
6. Marketing
7. People
8. Product
9. Sales
10. Security

</details>

<details markdown=1>

<summary>BIOps Roles and Responsibilities</summary>

Please see the project-permission-structure section for details on the permissions for the BIOps roles.

1. Tableau Admins / Maintainer Responsibilities: These leaders are responsible for publishing content in the Sub-Projects that role up to the Top Level Projects and are responsible for maintaining the GitLab Tableau Project. This role does not specifically include Tableau Cloud Site Administration responsibilities although several Top Level Project Leaders are also Tableau Cloud Site Admins.
2. Project Leader / Code Owner Responsibilities: Project Leaders come from functional departments and teams. These leaders are responsible for reviewing and approving content for publishing in their department's folder and in cross-functional Sub-Project folders like the Go To Market folder as Code Owners. A full list of Project Leads can be found [here](#).

</details>

Tableau Content Publication

Tableau Workbooks and Data Sources can be published to two folder environments: Development and Production. The following details the publication process for each folder environment.

Publication to Development Folders

All Tableau Content Development starts in the Development Project Folder. The Development Folder is a Sandbox environment where Tableau developers are free to experiment and iterate with content and share with team members for initial peer reviews. Tableau Developers can organize their Sandbox work using Collections for easy access and sharing. Tableau Creators should follow SAFE development workflow guidelines when working with sensitive SAFE/PII/MNPI data in the Development folders.

Publication to Production Requirements

The Production Folder is a separate environment where Tableau users can find dashboards that have been peer-reviewed and tested by project leads of the respective functional areas. Content in the Production folders has met the following requirements:

1. Documentation

- Issue Creation: Publication of content requires an issue which is automatically labeled for categorization and search-ability.
 - Documentation in Handbook: Documentation to be created in GitLab Handbook and linked to in Tableau content description and listed on dashboards
1. Validation Approvals
 - Tableau Project Lead (Business Department Owner) Approval: The Tableau Project Lead must approve the promotion.
 - Technical Owner (Data Steward) Sign-off: Project Lead may request a Technical Owner sign off on the validity of the data and content.
 - Cross-Division Development: Content that is cross-division requires a review by multiple Business Owners. Cross Division content includes GTM, etc.
 1. Data Security
 - Data Security: For SAFE or Restricted data, appropriate folder and/or data source access permission controls must be applied. For general access data, confirmation that no restricted (SAFE) data is used is required.
 1. Uniqueness
 - Content Uniqueness: Content is unique to Production
 1. Requirements for Workbook Formatting
 - Workbook Formatting: The workbook must include the GitLab logo.
 - Content Descriptions: Workbooks and Data Sources should have a 1-2 concise description providing the purpose of the content.

Publication to Production Procedure

First-Time Promotion to Production

1. Developer Publish to Development: The Developer publishes the content to a Tableau Development folder. This is the initial step where content is prepared for review and eventual promotion.
1. Developer Creates an Issue: The Developer creates a GitLab Issue which will document the promotion process to Production. Use the template Tableau Publish Workbook or Data Source to Production which lists all necessary request documentation and requirements.
1. Developer Confirms Content Validity: Developer ensures content meets promotion criteria requirements as documented in Issue. The Developer will then assign the issue to the associated Department Project Lead for approval, signaling that the content is ready for review.
1. Review Process: The Project Lead will review the issue and content. They will check for compliance with Production requirements and assess the content's readiness. If needed, the Approvers will propose corrections or updates to the content. The Developer will need to address these before proceeding.
1. Final Promotion: Once all requirements are met and approval is granted, the Project Lead moves the content from Development to Production. This step officially makes the content available for business use.

Editing and Updating Content in Production

There are two options for editing or updating content which is in Production:

1. Update and Overwrite Original: This is required for any major changes to a workbook or data

source. Changes include replacing or editing a data source, altering or adding logic, custom sql, or calculated fields. This is the preferred option where the developer edits the workbook or data source and save it to the Development folder. It is then reviewed and inspected. After approval it the Project Leader will overwrite it to Production.

1. Allow Developer to Edit in Production: Allow me 48 hours to edit/update access to alter in Production content myself. This is only allowed for small changes such as cosmetic improvements, spelling corrections, small filter changes or urgent issues.

Tableau Style Guide

For more guidance on design best practices and resources, please refer to our Tableau Style Guide handbook page.

Tableau Publishing Service Level Objectives (SLOs)

Production Publishing SLO

Project Leaders are responsible for their respective department folders. They provide review and approvals for requests to promote content to Production.

Development Publishing SLO

Individual Tableau developers can publish development content on-demand to their department's development sub-project.

Deployment

Tableau Cloud leverages GitLab's existing technology investments and integrates into our IT infrastructure to provide a self-service, modern analytics platform for our users.

Tableau Unused Content Archival Policy and Process

To maintain an efficient, clean, and user-friendly Tableau environment an automated archival process has been implemented. This policy outlines the procedure for archiving unused and inactive content within our Tableau GitLab Cloud environment. Tableau GitLab Public and Sandbox are excluded from this policy.

Scope of Archiving:

90+ days unused workbooks and data sources are archived and made inaccessible by search or via direct links. Archived items can be retrieved upon request to the BI team. The archival process is automated and occurs on the 15th of each month by moving content to the Admin Archive Folder.

Archival Exclusions:

1. Workbooks and dependent data sources published to Tableau GitLab Public.
1. Items for which the owner has requested retention, indicated by the Tableau tag Do NOT Archive.
1. Content stored in Personal Folders will not be archived, as it is only viewable by the owner

and does not clutter the environment.

Archival Contact:

For questions or requests regarding archived content, please reach out to the BI team. [Click here](#) for administrative archival documentation.

Permissions

Pursuant to GitLab's Transparency value, all GitLab team members will have access to all content in Tableau by default. However, as a Public Company, we have to abide by the SAFE Framework and limit access to certain sensitive and confidential data in Tableau to team members that need SAFE data to do their jobs. In Tableau, we set these SAFE data permissions via User Groups at the Project level.

User Groups

User Groups are the only prescribed method we use for setting permissions across the Tableau site. A User Group is a collection of users that can be based on a topic, project, or organization structure, that will need to have the same set of access and permissions for content. All users will be a member of the General Access user group and can be added to more User Groups as required.

User groups will allow business teams to manage accessibility to their published content based on rules that they've identified. Request for the creation of a limited access user group can be made through the Standard Data Team Issue template in Issues section of the Tableau project.

List of User Groups

Each section below corresponds to a limited access user group and the designated owner. Access to limited user groups can be done via the Lumos app using the Tableau SAFE Access or Tableau Special Permissions applets. Please note: To gain access to an user group, the designated owner will need to give approval. The designated owner can be seen in Lumos when requesting access.

- \General SAFE Access: This group allows viewing of and development with data that contains material non-public information that should be kept SAFE. Team members must be on the Designated Insiders list to be added to this group.

- To gain access to SAFE data and be part of the SAFE Access group please submit a request through Lumos.

- Ecosystem Sales and Operations

- Internal Audit Restricted Access

- People Restricted Access

Project Permission Structure

User Groups are applied at the Project and Sub-Project levels. We have two types of User Groups that we apply, Administrator User Groups and Access Control User Groups. The Administrator User Group gives permissions relating to what administrative tasks the user can do on the project

such as moving content, deleting content, and changing permissions on the project. The Access Control User Group gives permissions relating to what access the user has such as viewing and publishing content.

The Administrator User Group and the Access Control User Group can be customized to meet the unique needs and requirements of each Project and Sub-Project on the Tableau site. This allows for flexibility to add or customize security controls on an as-needed basis. The assignment of a User Groups permission rule set for a Project or Sub-Project will be documented and controlled from YAML files maintained in the Tableau Project in the Data Group.

No content is published in the top level Production, and Development Projects. Content is only published in the Sub-Projects under the Top level projects which is a best practice for content, access, and security control. Top level Project (Production and Development) Leaders will always be a member of the Central Data Team's BI Platform team and will need a site role of Creator to function as Project Administrators for publishing content. Project Leaders own the Sub-Projects for their respective functional areas. The Project Leaders will be added to the applicable Administrator and Access Control Groups in order to have the right permissions to lead the Sub-Project.

The standard permission rules for top level Projects are noted below:

Project	View	Publish	Administer
Development	All Team Members	All Team Members	Project and Sub Project Leaders
Production	All Team Members	Project Leaders	Project Leaders

Below is an example of User Groups and Permissions applied to a Data Team Sub-Project where only Data Team Members can publish in the project, but All Team Members can view the content in the Sub-Project. At the Sub-Project Level, for the User Group Name, Limited Access Team Members can replace the All Team Members User Group Name for limited access.

User Group Name	User Group Type	Permission Template
BI Platform Team	Administrator Group	Inherited Administer
Data Team Project Leaders	Administrator Group	Administer
Data Team	Access Control Group	Publish
All Team Members	Access Control Group	View

Capabilities

Capabilities are a Tableau concept that we apply. Permissions are made up of capabilities, or the ability to perform a given action on a piece of content, such as view, filter, download, or delete. Permission rules are the setting for each capability (allowed, denied, or unspecified) for the user group. Permission rules have templates available that make it easier to assign capabilities quickly.

Templates group sets of capabilities that are often assigned together based on common user scenarios such as View, Publish, and Administer. When we assign a template, its included capabilities are set to Allowed, with the rest left as Unspecified. The templates are built to have all of the capabilities a user would need, for example the Publish template includes

everything from the View template plus additional capabilities.

Below are examples of Permission Templates we use at GitLab. Permission templates are applied to Tableau content.

Capability	View	Publish	Administer
View/Interact/Use	X	X	X
View/Add Comments	X	X	X
Download Image/PDF	X	X	X
Share Views	X	X	X
Download Data		X	X
Web Edit		X	X
Overwrite/Publish		X	X
Move			X
Delete			X
Change Permissions			X

Below are examples of Site Roles that we use. Individual users are assigned to site roles and given permissions via site role. Site roles act as an upper limit of what a Tableau User can do on the site.

Capability	Viewer	Explorer	Creator
View/Interact	X	X	X
View/Add Comments	X	X	X
Download Image/PDF	X	X	X
Download Summary Data	X	X	X
Download Full Data		X	X
Share Views		X	X
Web Edit		X	X
Overwrite/Publish		X	X
Move		X	X
Delete		X	X
Change Permissions		X	X
Create Alerts		X	X
Publish Data Source			X
Use Data Flows			X
Site Administration			X

<details markdown=1>

<summary>Example Permission Templates YAML File</summary>

yml

permissiontemplates:

- name: view

- projects:

- Read: Allow View

workbooks:

Read: Allow View

Filter: Allow Filter

ViewComments: Allow View Comments

AddComment: Allow Add Comments

ExportImage: Allow Download Image/PDF

ViewUnderlyingData: Allow Download Summary Data

ShareView: Allow Share Customized

datasources:

Read: Allow View

Connect: Allow Connect

ExportXml: Allow Download Data Source

dataroles:

Read: Allow View

flows:

Read: Allow

ExportXml: Allow Download Flow

lenses:

Read: Allow View

metrics:

Read: Allow View

virtualconnections:

Read: Allow View

Connect: Allow Connect

databases:

Read: Allow View

tables:

Read: Allow View

- name: projectlead

projects:

ProjectLeader: Allow Set Project Leader

</details>

<details markdown=1>

<summary>Permissions Best Practices for Admins & Project Leaders</summary>

- Don't publish in the parent folder(s) and create nested subfolders instead.
- Set permission on a project folder level, not an individual workbook.
- Assign permissions to a group, not an individual person.
- Required course to complete: Site Management
- Ensure non-SAFE users cannot access SAFE content. This is the only time you should make use of the "deny" permissions functionality.

</details>

Tableau License Management

Our organization manages Tableau licenses through an automated system using an Okta - Lumos integration. This approach ensures everyone who needs Tableau has access to it by redirecting unused licenses from inactive accounts to team members who will benefit from the platform's insights and capabilities.

License Activity Policy

To maintain a Tableau license, users must actively use the platform within a 90 day period. Our automated Lumos system monitors login activity and manages licenses based on usage:

- Users who don't log in to Tableau within 90 days may be automatically deprovisioned
- Tableau Viewer licenses are automatically removed after 90 days of inactivity without additional review
- Tableau Creators and Explorers undergo a review by the Tableau Administration team before deprovisioning
- Note: VP-level positions and above are exempt from automated deprovisioning

Regaining Access

If deprovisioned due to inactivity:

1. Access Lumos portal through Okta
2. Submit Tableau access request for Tableau Access
3. Upon approval, access will be restored

Permissions Preservation: When a user is deprovisioned, they only lose their license not their group memberships or permission settings. This ensures that when access is restored, users automatically regain access to all their previous projects, dashboards, and content without needing additional configuration.

This approach ensures licenses are available for active users while allowing easy restoration of access when needed and follows our Data Health and Security practices.

In addition to the guidelines above, we typically reserve Creator licenses for team members whose role is primarily Analytics-based (i.e. analysts within functional teams, and/or team members whose core responsibilities include developing reports for their team).

Tracking License Usage

- Utilization & Usage by License Type
- Assigned Licenses (Admins only)
- Licenses Used
- Actions by Users
- eLearning Usage (Admins only)

Access

Getting Started

Users can request access to Tableau through the Okta portal using the Lumos application. Follow these steps to submit your request for Tableau:

- Log in to the Okta portal
- Search for and select the "Lumos" application
- Select "Tableau" from the available applications
- Choose the specific role that matches your needs (see Role Types below)
- Lumos will automatically initiate the approval workflow and handle provisioning upon approval

The system will guide you through any additional information required to complete your request. You'll receive notifications about the status of your request through email.

Role Types

When requesting access, select the appropriate role based on your needs:

- Viewer: View and interact with dashboards and visualizations
- Explorer: Create and modify workbooks from existing data sources
- Creator: Build new data sources and develop advanced visualizations

[Click here](#) for further details on role capabilities.

Restricted Data Access

Tableau access is structured in a tiered approach that separates data based on sensitivity levels and regulatory requirements.

- General Content: All users receive access to general content by default upon approval
- Restricted SAFE Content: Requires additional approval and justification
- Other special permission access.

Users may request restricted access through the Lumos App by selecting the following Lumos Apps and following the instructions there:

- Tableau Restricted SAFE Access
- Tableau Special Permissions

Tableau Desktop Access

Creators with an active license to Tableau Cloud are encouraged to use Tableau Desktop for development. Locally developed Data Sources or Workbooks can later be published to Tableau Cloud. All Creators will be assigned access to Tableau Cloud and Desktop. Tableau Desktop keys are automatically assigned by Tableau Cloud when first connecting.

Tableau Creators who use Tableau Desktop will need a Yubikey set up in Okta to access content published in Tableau Cloud. If you're unsure which one to get, the recommended device is the YubiKey 5C Nano FIPS. Currently biometrics are not yet supported in Tableau Desktop.

One can download Tableau Desktop using the links below, or follow the link from the Home Page of Tableau Cloud.

- [Tableau Desktop Releases Download](#)
- [Tableau Prep Builder Releases Download](#)

If you want to automatically download the latest version of Tableau Desktop for Mac, you can use this link.

<details markdown=1>

<summary>Updating Tableau Desktop</summary>

To update Tableau Desktop to the latest release, follow these steps:

1. Download the Latest Version: Visit [Tableau's Download Page](#) and download the latest version of Tableau Desktop.

1. Install the Latest Version: Run the downloaded installer and follow the on-screen instructions to complete the installation of the latest version of Tableau Desktop.

1. Uninstall Previous Versions: To maintain a clean environment and reduce any version conflicts, uninstall any previous versions of Tableau Cloud applications that may still be on your device. This can be done by closing all Tableau Desktop instances, open applications folder, locate previous versions of Tableau Desktop, and then moving them to trash.

</details>

Tableau Desktop vs. Tableau Cloud

Tableau offers two ways to create / edit workbooks and data sources, either via the Desktop client or Cloud (web) version. However, we recommend developing in Tableau Desktop, as this version will have full functionality for editing and development.

Benefit of each version:

Desktop

Tableau Desktop provides FULL functionality. All the capabilities available by Tableau will be accessible in Desktop and you can create data sources, workbooks, dashboards, groups, sets, formatting, customized calculations, etc.

Cloud

Tableau Cloud allows users to interact with content (that you've created and published with Desktop). Cloud will also allow you to create workbooks / dashboards, but not data sources, sets, in-depth formatting, etc.

For more details on features offered by each version please see this [Tableau article on Web Authoring and Tableau Desktop Feature Comparison](#)

Tableau Data Sources

Tableau Data Source Connection Policy

This policy provides guidance on approved data source connections in Tableau to ensure data consistency, security, and support across all dashboards.

1. Primary Data Source: Snowflake

- Role: Snowflake is the main data source, housing core business logic and transformed data. All data requiring transformations or business rules must originate from Snowflake.
- Support: The Tableau Admin team fully supports Snowflake connections in Tableau.

2. Additional Approved Data Sources

- Permitted Sources: Google Sheets, Google Drive, Excel are permitted for external data that doesn't duplicate Snowflake data.
- Support: The Tableau Admin team will provide support for the listed permitted connections in Tableau.
- Policy Compliance: The connection must meet policy guidelines, avoiding duplication of Snowflake data and excluding additional business logic or transformations that belong in the warehouse pipeline.

3. Additional Data Connections and Available Connectors

Tableau provides a wide range of Supported Connectors and Tableau Exchange Connectors. Generally, all data should be routed through the approved data pipeline. However, direct connections to alternate data sources are permitted in specific cases, such as when data does not require merging with business logic, and a direct connection provides efficiency and quicker time implementation.

These direct connections may only be used under the following policy:

- Approvals: The connection must be approved by the Data Team and source system business and technical owner. Start a new request [here](#).
- Security: The connection must be secure. A security audit may be required.
- Policy Compliance: The connection must meet policy guidelines, avoiding duplication of Snowflake data and excluding additional business logic or transformations that belong in the warehouse pipeline.
- Data Privacy and Governance: Data access should be restricted as necessary to protect sensitive information, such as PII and MNPI. Coordinate with the Tableau Admin Team to ensure proper data security and privacy measures are in place.

Data Source Access: Tableau Cloud

Please refer to this [Connecting to Data in Tableau Guide](#) for more details.

<details markdown=1>

<summary>Snowflake</summary>

In order to use the Snowflake connector, you must have a Snowflake account assigned by the Data Platform team. Please open an AR ([example](#)), if you do not already have access.

<summary>Data Sources published in Tableau Cloud</summary>

1. Personal Space > New > Workbook
2. Connect to Data > On This Site
3. Pick a published data source

</details>

<details markdown=1>

<summary>Flat files (formats: xls/xlsx, csv, tsv, kml, geojson, topojson, json)</summary>

1. Personal Space > New > Workbook
2. Connect to Data > Files
3. Drag and drop a file / Upload from computer

</details>

<details markdown=1>

<summary>Google Sheets</summary>

1. Personal Space > New > Workbook
2. Connect to Data > Connectors > Google Drive
3. Sign in with your Google Account
4. Check all the boxes
5. Click Allow
6. Double click on the Google Sheet you want to use

</details>

Data Source Access: Tableau Desktop or Tableau Prep Builder

Important: In order to connect Tableau Desktop to Tableau Cloud, you need to set up a Yubikey in Okta. Fingerprints will not work. Please see the Tableau Cloud Access section above for more details on how to order a Yubikey.

<details markdown=1>

<summary>Data Sources published in Tableau Cloud</summary>

1. Connect > Search for Data > Tableau Server
1. Quick Connect > Tableau Cloud
1. Log in to Okta - if Okta isn't set up for your account, login using your Tableau login & MFA
1. Pick a published data source

</details>

<details markdown=1>

<summary>Flat files (formats: xls/xlsx, csv, tsv, kml, geojson, topojson, json)</summary>

1. Connect > To a File
1. Select a data file format

</details>

<details markdown=1>

<summary>Google Sheets</summary>

1. Connect > To a Server > Google Drive
1. Sign in with your Google account
1. Check all the boxes
1. Click Allow
1. Click on the Google Sheet you want to use
1. Click Connect

</details>

Snowflake OAuth Data Source Connection Expiration Period

Snowflake credentials in Tableau data sources are valid for 90 days. They reset automatically whenever new content is published to Snowflake-connected workbooks or data sources. However, if no publishing occurs within 90 days, the credentials will expire, requiring manual reauthentication. Click the below drop down for the process to update the OAuth Credentials:

<details markdown=1>

<summary>Updating Your Snowflake Credentials in Tableau</summary>

1. Access Account Settings:
 - Log in to Tableau.
 - Click your profile icon in the top-right corner of the screen.
 - Select Account Settings from the dropdown menu.
1. Locate Snowflake Credentials:
 - Scroll down to the Snowflake section to view your current credentials.
1. Test Each Entry:
 - For each Snowflake entry listed, click Test.
 - If the test fails, proceed to the next step.
1. Remove and Re-add Credentials:
 - Delete the failed Snowflake entry by clicking Delete.
 - Click Add to re-add your credentials.
1. Re-add Credentials:

- Select OAuth Credential as the authentication method.
- Enter the Snowflake Server under OAuth instance.
- Leave the Role field blank.

1. Complete Authorization:

- A new window will pop up.
- Click Single Sign-On (SSO)
- Click Authorize to complete the process.

1. Repeat for All Accounts:

- Perform these steps for each Snowflake account you have set up.

By following these steps, you'll ensure that your Snowflake OAuth tokens are updated and prevent credential expiration from impacting your Tableau workflows.

</details>

Tableau Certified Data Sources

Certified data sources ensure trusted, high-quality, and efficient data for the organization. Approved by a Tableau Admin through an issue submission, these sources receive a Certified badge and an appended " - Certified" name for visibility and searchability. The certification signals that the data source has been rigorously reviewed for accuracy and compliance, providing users confidence to build reliable reports.

Certified data sources are designed to be the most commonly used single sources of truth, aligned with the GitLab Data Development lifecycle. They are broad, reusable, and versatile, capable of addressing a wide range of analytical needs. A typical department may maintain 2-12 certified data sources, serving as the primary resources for high-quality and efficient development. Departments retain ownership of these sources, with Project Leaders actively overseeing their maintenance to ensure they remain reliable, foundational tools for the organization. To avoid overly complex, multifunctional data sources, multiple Certified, fit-for-purpose data sources are invited.

As the primary resource for analysts, certified data sources streamline onboarding, training, and updates while reducing duplication and promoting consistency. By focusing on quality and reliability, they simplify decision-making and solidify their role as essential components of the organization's data ecosystem.

Recommend we allow best in class BI standards, plus modeling standards, fit for purpose approaches to use cases, and usability for the end users driving the build.

Tableau Certified Data Source Policy

All certified data sources must meet the following criteria:

1. Data Accuracy and Quality: Data must be validated against source systems to ensure accuracy while leveraging and inheriting the quality standards established by the Enterprise Data Model (EDM). Additionally, Tableau relationships and joins, filters, and calculated fields should be reviewed for correctness and consistency. Key metrics and calculations must be verified not

only for consistency with source systems but also for alignment with the EDM-s defined structures and principles.

1. Documentation: Tableau Data Source should contain the following where it can be picked up by the Data Dictionary:

- Descriptive Name: Clear and user-friendly
- Description: Include purpose, data grain, and DRI ownership

1. Performance: Data should load in a reasonable amount of time. Queries should be efficient and optimized for extracts or live connections.

1. Data Security and Compliance: Data should adhere to data security, privacy, and governance policies including SAFE access. Ensure restricted content is published to the appropriate project folder with row-level security applied where necessary to safeguard sensitive data.

1. Custom SQL in Data Sources: Data sources that use custom SQL may be certified if the custom SQL is being used for specific use cases, including: Row-level security, custom parameters such as query size limit filtering, and complex joins. Including custom SQL to apply business logic (i.e. alter calculations or conditions) will block a Tableau data source from being certified. This is because we aim to keep business logic within the transformation/data warehouse layer so that it is under source control, can be easily contributed to via an MR, and produces the same results across both the data warehouse & the BI layer.

1. Field Naming and Formatting:

- Use intuitive names (e.g., ·Transaction Date· instead of ·txndt·)
- Avoid Redundancy: Eliminate prefixes/suffixes that repeat context already provided (e.g., ·Region· instead of ·SalesRegionRegion·).
- Hierarchical Data: Use Tableau levels for hierarchies (e.g., Country · State · City) to enable drill-down functionality.
- Consistent Case and Spacing: Use proper case and spaces instead of underscores (e.g., ·Customer Name· instead of ·customername·).
- Ensure correct data types and formatting:
 - Dates: Convert dates to ISO:yyyy/mm/dd or change datetime to date when time precision isn't required.
 - Numbers: Format as currency or decimal where needed.
 - Percentages: Apply proper '%' formatting at desired decimal level.

Data Team Tableau Resources

The Data Team provides shared Tableau resources to support reporting, dashboard development, and data visibility across the organization. These resources are maintained to help teams build with consistency, understand platform usage, and access historical content when needed.

You can find these resources in the Production · Data Team · General folder. This location includes information such as:

- Platform Usage Dashboards · Insights into how Tableau is being used across teams
- Archived Dashboards and Data Sources · Retired or historical content for reference
- Templates and Examples · Standardized templates to help jump-start new dashboard builds

Use this folder as your go-to spot for foundational Tableau materials supported by the Data Team.

Education

GitLab team members who realize the full potential of analytical insights can do powerful things with data. But having a platform like Tableau and access to data isn't enough; we need to assure that our users are prepared to use Tableau effectively.

Official Training Resources

- Free Tableau Training
- Tableau eLearning - Training Portal: eLearning from Tableau is available for users with the Creator or Explorer license type
 - Tableau eLearning access steps
 1. Go to explorer-elearning.tableau.com
 1. Create (or login to) your TableauID account
 1. Confirm your email address via the TableauID confirmation email
 1. Go to <https://explorer-elearning.tableau.com> and log on using your TableauID
 1. Register at the Tableau Learning Center with the Access Code available [here](#).
 1. Start using eLearning by clicking on a Course or Learning Path.
 1. Return to <https://explorer-elearning.tableau.com> to continue using Tableau eLearning.
- If you experience any issues accessing the training content, check this page for solutions to the most common problems.
- Tableau Community
- Tableau Support
- Tableau Classroom training & Training Pass
 - These could be options for you to use as part of your growth and development benefit. Bring this up with your manager during your career development conversations.

<summary>Third Party Training Resources</summary>

- YouTube Intellipaat - Tableau Cloud Training
- YouTube edureka! - Tableau Full Course
- YouTube Simplilearn - Tableau Tutorial

Note: training videos listed above are provided for free by third parties and their content has not been fully vetted by either Tableau or the GitLab Data Team.

</details>

<details markdown=1>

<summary>Recommended Training</summary>

Below is the recommended training course(s) and required time estimation for the following roles based on their responsibilities and required skill sets. For the full list of courses and learning paths please see our handbook section under Training by Roles:

Site Administrator

- Minimum: Introduction to Site Administration (time estimate: 3 hours)

Project Lead / Maintainer / Code Owner

- Minimum: Getting Started with Tableau Cloud (time estimate: 4 hours)

Creator / Developer / Analyst

- Minimum: Getting Started with Tableau Desktop (time estimate: 4 hours)

Explorer

- Minimum: Get Started with Tableau (time estimate: 4 hours)

Business Owner / Stakeholder / Viewer / End User

- Minimum: TBD (time estimate:)

</details>

<details markdown=1>

<summary>Training by Roles</summary>

Below is a set of training courses and learning paths that are recommended for the following roles based on their responsibilities and required skill sets:

Site Administrator

- Site Administrator Learning Path (estimated)
- Tableau Services & Support

Project Lead / Maintainer

- New Feature Spotlight Course
- Community Leader Learning Path
- Tableau Intermediate Course
- Tableau Advanced Course

Creator / Analyst

- Author Learning Path
- Designer Learning Path
- Analyst Learning Path
- Data Scientist Learning Path
- Developer Learning Path
- Tableau Fundamentals Course
- Tableau Prep Builder Course

Business Owner / Stakeholder

- Executive Sponsor Learning Path

End User

- Consumer Learning Path

</details>

<details markdown=1>

<summary>Tableau Certification</summary>

Tableau offers several certifications for users to prove their specialized skills in and knowledge of the tool. The following certifications are recommended for creators / analysts who want to showcase their expertise:

- Certified Tableau Desktop Specialist: This exam is for those who have foundational sk